

LLMVul: A Vulnerability-Labeled Dataset of LLM-Generated C/C++ Functions from Real Production Repositories

Mohammad Farhad

School of Computing & Informatics
University of Louisiana at Lafayette
Lafayette, LA, USA
mohammad.farhad1@louisiana.edu

Shuvalaxmi Dass

School of Computing & Informatics
University of Louisiana at Lafayette
Lafayette, LA, USA
shuvalaxmi.dass@louisiana.edu

Abstract

Large language models (LLMs) are increasingly used to generate and assist with software development, yet existing vulnerability datasets largely focus on human-written code or controlled prompting environments. This limits the ability to study security weaknesses in LLM-generated code as it appears in real-world software projects. We present LLMVul, a vulnerability-labeled dataset of LLM-generated C/C++ functions mined from real production repositories. We mine AI-assisted development activity from GitHub over a 4 year period, from November 13, 2022 to September 3, 2026, using provenance signals such as commit metadata and AI-related authorship evidence. After filtering and deduplication, LLMVul contains 21,430 unique C/C++ functions from 226 repositories, together with repository, commit, function, provenance, and AI-tool metadata. We establish vulnerability labels using an ensemble of complementary static-analysis and pattern-based techniques and assign Common Weakness Enumeration (CWE) categories to confirmed vulnerable functions. To assess labeling reliability, we additionally conduct independent manual annotation and measure inter-rater agreement using Cohen’s kappa ($k = 0.79$). LLMVul contains 1,540 ensemble-vulnerable functions spanning 17 unique CWE categories, providing substantially more real-world LLM-generated vulnerable C/C++ functions than existing vulnerability-oriented LLM code benchmarks. By preserving both code-level vulnerability labels and generation/provenance metadata, LLMVul enables reproducible research on vulnerability detection, security evaluation of LLM-generated code, and analysis of vulnerability patterns in AI-assisted software development. The LLMVul dataset is publicly available at <https://doi.org/10.5281/zenodo.22668216>.

CCS Concepts

• **Security and privacy** → **Software security and testing; Vulnerability analysis; Software engineering;**

Keywords

Source Code Analysis, Vulnerability Detection, LLM-generated Code, Dataset, Software Security, Mining Software Repositories

1 INTRODUCTION

The widespread adoption of large language model (LLM)-based coding assistants has fundamentally altered how software is produced. GitHub Copilot surpassed one million active users within months of its general availability in June 2022 [8], and by 2024 reported over 1.8 million paid subscribers [9]. Tools such as ChatGPT, Claude, Cursor, and Google Gemini are now routinely used to

generate production code across security-sensitive domains including systems software, embedded firmware, and network services [22]. Recent large-scale measurements confirm that approximately 20–30% of new commits in major open-source repositories contain AI-attributed code [14, 22], with projections suggesting this proportion will continue to grow [1].

This shift introduces a security risk that the research community has only begun to examine. LLM-generated code is known to exhibit elevated vulnerability rates: Siddiq and Santos [19] found that 74% of Copilot-generated Python functions contained security weaknesses detectable by human experts, while Perry et al. [17] showed that developers who used AI assistance were *significantly more likely* to produce insecure code than those who did not. A 2026 formal verification study of 3,500 LLM-generated artifacts reported that six industry-standard static analysis tools combined detected only 7.6% of formally proven vulnerabilities, missing 97.8% of exploitable code [3]. Most recently, SecRepoBench demonstrated that GPT-5—the most capable model currently available—achieves only 39.3% secure-pass@1 on real-repository code completion tasks [18], confirming that the security problem persists even with the best available models and agent frameworks.

The dataset gap. Despite this growing body of evidence, the datasets available to the research community for studying LLM-generated code security share a critical structural limitation: *existing benchmark mostly rely on controlled, researcher-directed prompts rather than from code that developers actually committed to production repositories*. SecurityEval [19] provides 130 hand-crafted prompts designed to elicit specific CWE patterns from code generation models. CyberSecEval 2 [2] and SafeGenBench [13] follow the same paradigm at larger scale—558 and approximately 500 samples respectively—while CWEval [16] deliberately isolates functions from third-party dependencies to simplify evaluation. As a consequence, existing LLM security benchmarks primarily measure how often LLMs generate specific vulnerabilities under explicit security-oriented prompts, capturing a controlled laboratory phenomenon rather than real-world developer behavior. In production, however, security vulnerabilities may emerge incidentally when developers use LLMs for routine feature implementation without any explicit security framing. This gap leaves an important question largely unanswered: *How frequently do LLMs introduce security vulnerabilities during ordinary, non-security-focused code generation?*

Two recent empirical studies have examined the security of LLM-generated code in real-world settings, yet neither provides labeled vulnerability data for systematic security analysis. Wang et al. [22] mine AI-attributed commits from 1,000 GitHub repositories and confirm that certain CWE families are overrepresented in AI-tagged

commits—but provide no *function-level* vulnerability labels suitable for training or evaluating detection models. Liu et al. [14] analyse technical debt in 3,02,600 AI-authored commits across 6,299 repositories but focus on Python, JavaScript, and TypeScript rather than the C/C++ ecosystem, and similarly provide no vulnerability labels. On the human-written side, datasets such as BigVul [7], Devign [25], and PrimeVul [6] provide high-quality labeled C/C++ vulnerability corpora, but are composed entirely of human-authored code. Recent work has demonstrated that models trained on these corpora may not generalise to LLM-generated code, whose vulnerability patterns differ structurally from human-written code [6].

Our work. We present **LLMVul**, the first vulnerability-labeled dataset of C/C++ functions mined from *real production repositories* (as projects shown in Table 1) where code was generated by AI coding assistants. LLMVul bridges the gap between laboratory benchmarks and real-world deployment: rather than prompting an LLM to produce vulnerable code, we identify functions that developers committed to GitHub after using AI coding assistants, including GitHub Copilot, ChatGPT, Claude Code, and Cursor, and assess their security using an ensemble of three static-analysis tools—Semgrep, Flawfinder, and pattern-matching. The resulting vulnerability labels were subsequently validated on a representative sample by human raters, achieving substantial inter-rater agreement ($\kappa = 0.79$). The dataset comprises 21.4k unique C/C++ functions drawn from 226 repositories and 1,684 unique LLM-attributed commits spanning November 2022 to September 2026, of which 1,540 (7.2%) are labeled vulnerable across 17 unique CWE categories. Each function is annotated with full commit provenance, AI tool attribution, individual tool verdicts, and CWE identifiers, enabling a range of research tasks described in Section 4.

2 RELATED WORK

Existing research on the security of AI-generated code encompasses three broad categories of datasets: human-written vulnerability benchmarks, LLM-prompted code datasets, and datasets mined from AI-assisted software development in public repositories. Building on this taxonomy, Table 9 compares LLMVul with representative existing benchmarks across key dimensions, highlighting differences in data provenance, code-generation context, vulnerability coverage, and labeling methodology.

3 DATASET CURATION

LLMVul is constructed through a three-phase pipeline: (1) mining LLM-attributed C/C++ commits from public GitHub repositories, (2) extracting function-level code units, and (3) labeling extracted functions using a three-tool static analysis ensemble validated by human raters. Table 4 summarizes all 34 features recorded per function.

3.1 Phase 1: Repository Mining and AI Attribution

Repository selection. We queried the GitHub REST API for public C and C++ repositories with at least 200 stars and a commit activity after June 2022—the general availability date of GitHub Copilot [8]. In total we scanned 1,200 repositories and 321k commits, identifying

Table 1: Top 10 repositories by number of extracted functions.

Rank	Repository	Functions
1	Serial-Studio/Serial-Studio	6,159
2	nature-lang/nature	1,132
3	microsoft/ebpf-for-windows	699
4	isl-org/Open3D	466
5	microsoft/onnxruntime	465
6	DarkFlippers/unleashed-firmware	460
7	DavidXanatos/TaskExplorer	452
8	carla-simulator/carla	446
9	microsoft/WSL	410
10	sqliteai/warp	378

Table 2: Distribution of extracted functions identified by AI coding tool.

AI Tool (Total 9)	Functions
Claude Code	14,221
GitHub Copilot	4,952
Claude	807
Gemini	386
Cursor	373
ChatGPT	298
Generic LLM	242
OpenAI Codex	135
Devin	16

7,018 LLM-attributed commits from 226 repositories containing C/C++ functions (Table 1 showing top 10 repositories).

AI attribution signals. A commit is classified as *LLM-attributed* if its message or metadata contains at least one of two categories of signal, following the attribution methodology of Liu et al. [14]:

- **Strong signals** (signal_strength = strong): explicit co-authorship tags injected by the AI tool itself into the Git trailer, e.g. Co-authored-by: GitHub Copilot, Co-authored-by: cursor-noreply, or Co-authored-by: claude-code. These tags are machine-generated and carry the highest attribution confidence.
- **Medium signals** (signal_strength = medium): developer authored phrases in the commit message, e.g. “generated with ChatGPT”, “copilot suggested”, “ai-assisted” or “via cursor AI”. These phrases are self-declared by the developer and are slightly less certain than co-authorship tags.

The pipeline identified 89.2% strong-signal and 10.8% medium-signal attributed functions, as illustrated in Figure 2. These signals were derived from textual indicators of AI-assisted code generation. In total, we compiled up to 47 regular-expression patterns to detect these indicators across nine AI coding assistants: GitHub Copilot, ChatGPT, Claude / Claude Code, Cursor, Gemini, Devin, and OpenAI Codex. Claude Code accounts for the largest share of extracted functions, followed by GitHub Copilot (see Table 2). Of the 321k commits scanned, 7,018 (2.2%) matched at least one attribution pattern, a proportion consistent with Wang et al. [22], who report approximately 2% AI-attributed commits in production repositories. Attribution metadata is preserved in the ai_tool, signal_strength, and signal_type columns.

3.2 Phase 2: Function Extraction

For each LLM-attributed commit we retrieved the file-level differences via the GitHub API and extracted only the *added or modified*

Table 3: Summary of the mined C and C++ functions including repository statistics, commit metrics, lines of code (LoC) constraints and language distribution.

Metric	Value
Total functions	21,430
Repositories Mined	1,200
LLM-attributed repositories	226
Total commits scanned	3,21,080
LLM commits found	7,018
Unique commits	1,684
Date range	Nov 13, 2022 — Sep 3, 2026 (4 Years)
Maximum LoC	150
Minimum LoC	5
Language	Functions
C++	12,546 (58.5%)
C	8,884 (41.5%)

lines (i.e. lines prefixed with + in the unified diff format). Function boundaries were identified using the *tree-sitter* incremental parser [4] with its C and C++ grammars, targeting `function_definition` AST nodes. We applied two filters: functions shorter than five lines or longer than 150 lines were discarded as either too trivial or too large for reliable function-level analysis. Figure 1 illustrates the distribution of function lengths, measured in lines of code (LoC), for the functions retained in the final dataset. Each retained function is stored with its source coordinates (`function_start_line`, `function_end_line`, `function_lines`), the containing file (`file_name`, `file_hash`), and full commit provenance (`commit_id`, `commit_url`, `commit_message`, `commit_date`). A 16-character hexadecimal `unique_id` ties each record to its exact extraction context.

After extraction and deduplication removing identical function bodies that appeared in multiple commits—21,430 unique functions remained. A detailed breakdown is provided in Table 3.

3.3 Phase 3: Vulnerability Labeling

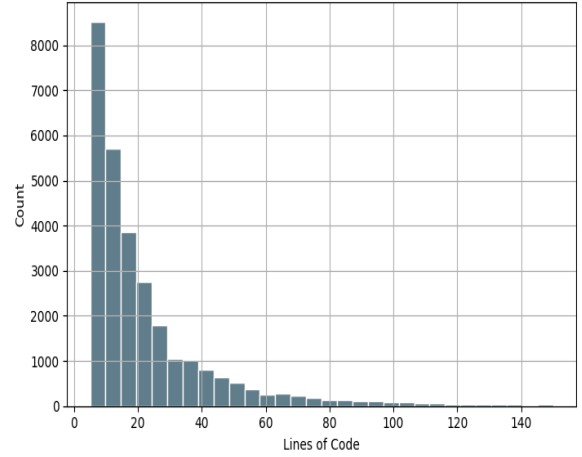
Three-tool ensemble. Each function was independently analysed by three complementary static analysis tools operating directly on the extracted function body:

1. **Semgrep** [11]: a pattern-matching SAST engine applied with CWE-based rules targeting dangerous function calls, format string sinks, and command injection patterns. Results are stored in `tool_semgrep` and `cwe_semgrep_raw`.

2. **Flawfinder** [23]: a lexical scanner that identifies calls to functions in its built-in risk database, assigning a risk level from 1 (lowest) to 5 (highest). Only findings at level 1 or above are retained. Results are stored in `tool_flawfinder`, `cwe_flawfinder_raw`, and `flawfinder_risk`.

3. **Pattern matching**: a curated set of 54 regular-expression patterns informed by the NIST Software Assurance Reference Dataset (SARD) dangerous-function taxonomy and the ITS4 vulnerability scanner [21]. Each pattern maps directly to a CWE identifier. Results are stored in `pattern_match` and `cwe_pattern_match_raw`.

Majority-vote ensemble. A function is assigned `vuln_label = 1` (*vulnerable*) if at least two of the three tools flag it; `vuln_label = 0` (*safe*) if none flags it; and `ensemble_label = uncertain` if only one tool flags it. Uncertain functions are excluded from the labeled training set but retained in the dataset for exploratory analysis. The final CWE identifier (`cwe_id`) is assigned by priority — Semgrep >

**Figure 1: Lines of code (LoC) distribution in the dataset.**

Flawfinder > pattern matching – preferring to prioritize the most specific tool-supported CWE over the generic CWE-676 fallback. CWE descriptions and CWE reference URLs are appended from the MITRE CWE catalogue [5] and stored in `cwe_description` and `cwe_url`.

Manual validation and inter-rater agreement. To assess label reliability, two independent raters (the authors) manually inspected a stratified random sample of 100 ensemble-vulnerable functions, drawn proportionally across CWE categories. Each rater independently assigned a binary label (1 = vulnerable, 0 = false positive) based solely on the function body and the assigned CWE identifier, without knowledge of the other rater’s labels or the tool outputs. Inter-rater agreement yielded Cohen’s $\kappa = 0.79$, indicating *substantial agreement* [12]. Disagreements were resolved through discussion and consensus; the consensus labels supersede the ensemble labels for the 100 validated functions.

Label distribution. Table 5 summarizes the outcome of the labeling process. Of the 21,430 functions, 1,540 (7.2%) are labeled vulnerable across eight CWE categories by the three-tool ensemble tool, with CWE-120 (Buffer Copy) and CWE-787 (Out-of-Bounds Write) being the most prevalent (see Table 6), and 17,211 (80.3%) are labeled safe, and 2,679 (12.5%) remain uncertain hence excluded from the binary-labeled set. The 7.2% observed vulnerability rate preserves the naturally occurring class distribution in our real-world sample, rather than artificially balancing vulnerable and non-vulnerable instances. Importantly, Flawfinder flagged most of the vulnerable functions as *dangerous function use*, that could not be precisely mapped to a more specific CWE. These cases were subsequently verified under the generic CWE-676 category, as shown in Table 7, with buffer-related issues constituting 84.6% of all classified findings. Table 8 reports the top ten projects with the highest concentration of vulnerable functions. Figure 3 visualizes the CWE distribution across these ten most vulnerable projects.

3.4 Data Quality and Reproducibility

Deduplication. Duplicate function bodies arising when the same AI suggestion appears in multiple commits or forks were identified

Table 4: Features and metadata contained in the LLMVul dataset.

Feature Category	Column Name	Description
Function Id	unique_id	Unique identifier for each function.
Project Name	project_name	GitHub repository containing the function.
Project URL	project_url	URL of the GitHub repository.
Commit Id	commit_id	Git commit identifier from which the function was extracted.
Commit URL	commit_url	URL of the corresponding commit.
Commit Message	commit_message	Commit message associated with the function.
Commit Date	commit_date	Date of the corresponding commit.
File Name	file_name	Source file containing the function.
File Hash	file_hash	Hash of the source file.
Source Code Language	language	Programming language (C/C++).
Function Information	function_body	Extracted source code of the function.
Function Startline	function_start_line	Starting line of the function.
Function Endline	function_end_line	Ending line of the function.
Function Length	function_lines	Number of lines in the function.
AI Provenance	ai_tool	AI coding assistant associated with the code.
AI Signal-Strength	signal_strength	Strength of evidence for AI-generated code.
AI Signal Type	signal_type	Type of evidence identifying AI-generated code.
Repository Stars	repo_stars	Number of repository stars.
Repository Forks	repo_forks	Number of repository forks.
Project Primary Lang.	repo_primary_language	Repository’s primary language.
Dataset Timestamp	mined_at	Timestamp when the function was collected.
Vulnerable Classification	vuln_label	Final vulnerability classification.
Ensemble Label	ensemble_label	Result of the ensemble analysis.
Semgrep Tool Results	tool_semgrep	Whether Semgrep flagged the function.
Flawfinder Tools Results	tool_flawfinder	Whether Flawfinder flagged the function.
Pattern Matching Results	pattern_match	Whether pattern matching flagged the function.
CWE Id	cwe_id	Final CWE assigned to the function.
CWE Information	cwe_description	Description of the final CWE.
CWE URL	cwe_url	CWE URL when a corresponding CVE is available.
Semgrep-Level CWE	cwe_semgrep_raw	CWE reported by Semgrep.
Flawfinder-Level CWE	cwe_flawfinder_raw	CWE obtained from Flawfinder.
Pattern-Level CWE	cwe_pattern-match_raw	CWE obtained from pattern matching.
Semgrep Metadata	semgrep_rule	Semgrep rule that triggered the finding.
Flawfinder Metadata	flawfinder_risk	Flawfinder risk level from 0 (lowest) to 5 (highest).

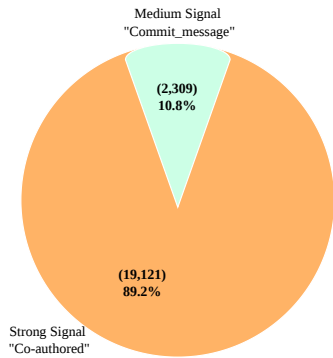


Figure 2: Signal strength identified by the pipeline.

Table 5: Summary of vulnerability classification ensemble and analysis results across the 21,430 functions.

Analysis Tools	Functions
Semgrep	2,459 (11.5%)
Flawfinder	2,091 (9.8%)
Pattern Matching	1,949 (9.1%)
Dataset Classification	Functions
Total functions	21,430
Vulnerable	1,540 (7.2%)
Safe	17,211 (80.3%)
Uncertain	2,679 (12.5%)

Table 6: Top CWE categories among the 1,540 ensemble-vulnerable functions.

CWE / Vulnerability Type	Vul. Functions
CWE-120: Buffer Copy	494
CWE-787: Out-of-Bounds Write	478
CWE-676: Dangerous Function Use*	411
CWE-22: Path Traversal	75
CWE-190: Integer Overflow	45
CWE-78: OS Command Injection	25
CWE-134: Format String	6
CWE-338: Weak PRNG	6
Total	1,540

*Dangerous functions includes CWE-369 (Divide by Zero), CWE-476 (Null Pointer Dereference), CWE-362 (Race Condition), CWE-416 (Use After Free), CWE-807 (Untrusted Inputs).

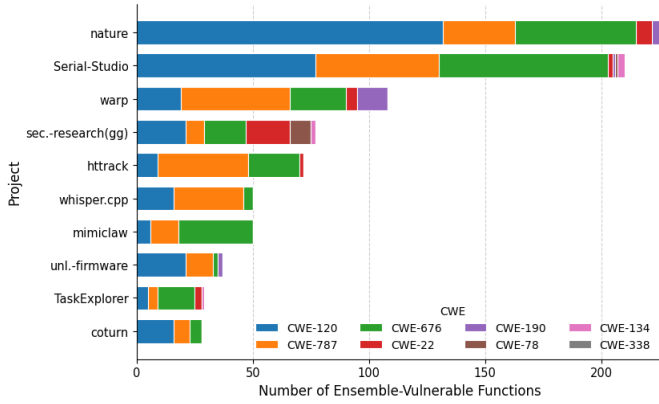
Table 7: Flawfinder finding distribution by category.

Category	Classified Findings
Buffer-related	1,307
Miscellaneous (e.g. divide by zero)	101
Formatted-I/O operations	41
Dangerous shell commands	30
Race-condition / TOCTOU-related	26
Integer-related	15
Unsafe temporary-file handling	13
Unsuitable random-number generation	6
Use of obsolete/deprecated functions	6
Total	1,545
Unclassified (but dangerous func.)	546

by exact SHA-256 content hashing and removed. Of the 29,062 raw

Table 8: Top 10 projects with the highest number of ensemble-vulnerable functions.

Project	Vuln. Func.	(%)
nature-lang/nature	227	14.74%
Serial-Studio/Serial-Studio	210	13.64%
sqliteai/warp	108	7.01%
google/security-research	77	5.00%
xroche/htrack	72	4.68%
ggml-org/whisper.cpp	50	3.25%
memovai/mimiclaw	50	3.25%
DarkFlippers/unleashed-firmware	37	2.40%
DavidXanatos/TaskExplorer	29	1.88%
coturn/coturn	28	1.82%

**Figure 3: CWE distribution across the top 10 projects with the highest number of ensemble-vulnerable functions.**

extracted functions, 7,632 (26.3%) were duplicates, yielding the final 21,430 unique records.

Repository diversity. To prevent any single large repository from dominating the dataset, we applied a initial cap of 300 commits per repository to ensure broad repository coverage while maintaining computational tractability. The 21,430 functions originate from 226 distinct repositories spanning 1,684 unique commits dated between November 2022 and September 2026.

Contamination awareness. Functions are associated with their `commit_date` to enable researchers to construct temporal splits and avoid data leakage when training models on pre-cutoff data and evaluating on post-cutoff functions—a practice recommended by Ding et al. [6].

Availability. LLMVul is publicly available on Zenodo under a Creative Commons Attribution 4.0 licence (DOI: <https://doi.org/10.5281/zenodo.22668216>). The mining and labeling scripts are released on GitHub at <https://github.com/Wahed08/LLMVul-Dataset>. A sample of 100 functions is included in the repository for immediate exploration without downloading the full dataset.

4 POSSIBLE RESEARCH QUESTIONS

LLMVul is designed to support empirical studies of the security, generalization, provenance, and evolution of LLM-generated C/C++ code. The dataset supports, among others, the following research questions.

Detection & Generalization.

RQ1: Do state-of-the-art vulnerability detectors trained on human-written code (e.g., LineVul, VulBERTa, and LLMxCPG) generalize to LLM-generated C/C++ functions, or do they exhibit significant F1 degradation relative to their performance on human-written code?

RQ2: Can a vulnerability detection model fine-tuned on LLMVul outperform general-purpose detection models trained on human-written code datasets such as Devign and PrimeVul when evaluated on LLM-generated code?

Security Characterization.

RQ3: Does LLM-generated C/C++ code exhibit a distinct CWE distribution from human-written vulnerable code in Devign and PrimeVul, and does this distribution vary across AI coding tools such as Copilot, ChatGPT, Claude, and Cursor?

RQ4: How has the proportion of vulnerable functions in LLM-generated C/C++ code evolved over time (2022–2026), and does it vary across AI coding tools and periods of use?

Dataset Utility & Analysis.

RQ5: Can LLMVul’s AI-attribution metadata (`ai_tool`, `signal_strength`, and `signal_type`) support function-level classification of LLM-generated versus human-written C/C++ code?

RQ6: How effectively do different components of the static-analysis ensemble (Semgrep, Flawfinder, and pattern matching) detect vulnerabilities in LLM-generated C/C++ code, and do their relative effectiveness rankings differ from those observed on human-written code benchmarks?

These research questions illustrate the utility of LLMVul beyond benchmark construction. Its real-world repository provenance, vulnerability labels, CWE information, AI-attribution signals, and temporal metadata enable systematic investigation of how LLM-generated code differs from human-written code and how existing security-analysis techniques generalize to this emerging code population.

5 THREATS TO VALIDITY

Several considerations regarding the scope and interpretation of LLMVul should be noted. First, vulnerability labels are generated using a three-tool static-analysis ensemble with a two-of-three majority-vote criterion. Static analysis may produce false positives or false negatives when applied to isolated function fragments. To assess the reliability of the labeling procedure, we manually validated a sample of 100 functions, obtaining ($\kappa = 0.79$). CodeQL was not included because its C/C++ extractor requires compilable translation units, which are incompatible with our function-level representation [10]. Second, AI attribution relies on explicit Git metadata signals. Consequently, AI-assisted functions for which developers did not record AI usage in commit metadata may not be identified, potentially causing LLMVul to underestimate the prevalence of AI-generated code. Moreover, substantial post-generation editing may make LLM-generated code less distinguishable from human-written code, which may limit the effectiveness of provenance-based analyses.

Finally, LLMVul samples the top 1,200 C/C++ repositories by star count to focus on widely used, actively maintained software projects. This sampling strategy favors mature and widely adopted projects and therefore may not fully represent smaller, private, less-established, or embedded codebases. LLMVul currently focuses on C and C++; whether the observed patterns generalize to other

Table 9: Comparison of LLMVul with representative human-written vulnerability, LLM-prompted, and AI-assisted code datasets. LLM-prompted datasets contain code generated through controlled research prompts, whereas AI-assisted datasets contain code produced with AI coding assistants during real-world software development and mined from public repositories.

Dataset, Year of Release	Purpose	Code Origin	Lang.	Repos	Unit	Scale	Vuln. Label	CWE Label	Ground-Truth Method	Prod. Code	AI Tool Meta
<i>— Human-written vulnerability datasets —</i>											
Devign [25], 2019	Vuln. detection	Human (CVE)	C/C++	2	Functions	27K+	✓	✗	CVE-linked	✓	✗
BigVul [7], 2020	Vuln. detection	Human (CVE)	C/C++	348	Functions	265K+	✓	✓	CVE-linked	✓	✗
ICVul [15], 2025	Vuln. detection	Human (CVE)	C/C++	807	Functions	15K+	✓	✓	Automated	✓	✗
PrimeVul [6], 2025	Vuln. detection	Human (CVE)	C/C++	755	Functions	235K+	✓	✓	Dedup+verif.	✓	✗
<i>— LLM prompted code datasets —</i>											
SecurityEval [19], 2022	LLM security eval	Prompted	Python	—	Prompts	130	✓	✓	Manual	✗	✗
FormAI [20], 2023	LLM vuln. analysis	Prompted	C	—	Programs	112K	✓	✗	Formal verif.	✗	✗
CyberSecEval 2 [2], 2024	LLM security eval	Prompted	Multi	—	Prompts	500	N/A	N/A	N/A	✗	✗
CWEval [16], 2025	Security + func. eval	Prompted	Multi	—	Functions	119	✓	✓	Dynamic	✗	✗
SafeGenBench [13], 2025	LLM security eval	Prompted	Multi	—	Functions	558	✓	✓	Automated	✗	✗
<i>— AI-assisted public code datasets —</i>											
DevGPT [24], 2024	AI-assist. dev. study	Real-world	Multi	—	Snippets	19K+	✗	✗	N/A	✓	Partial
Debt AI Boom [14], 2026	Tech. debt study	Real-world	Py/JS/TS	6,299	Commits	302.6K	✗	✗	N/A	✓	✓
LLMVul (Ours)	Vuln. detection for LLM code	Real-world	C/C++	1200	Functions	21.4K	✓	✓	3 Tool Ensemble + $\kappa=0.79$	✓	✓

Key: ✓ = available/yes; ✗ = not available/no; Prod. Code = mined from real production repositories (not controlled experiments); AI Tool Meta = records which AI coding assistant generated the code; Ground-Truth Method = methodology used to establish ground-truth labels; Py/JS/TS = Python, JavaScript, TypeScript only; Multi = multiple languages; — = not applicable (prompted datasets have no source repository).

programming languages, such as Python and JavaScript, remains an open question.

6 CONCLUSION & FUTURE WORK

We introduce LLMVul, a vulnerability dataset of C/C++ functions mined from authentic GitHub repositories containing code developed with AI coding assistants. Unlike controlled or simulated benchmarks, LLMVul captures vulnerabilities that arise incidentally during real-world AI-assisted development. The dataset comprises thousands of functions and commits, with vulnerabilities spanning multiple CWE categories and labels derived through a static-analysis ensemble and validated by human raters. Each function is accompanied by commit-level provenance and AI-tool attribution, enabling systematic investigation of the security, provenance, and generalization of LLM-generated code.

As future work, we plan to expand LLMVul to additional programming languages, including Python, Java, and Rust, enabling cross-language analysis of vulnerabilities in AI-generated code. We also plan to develop automated build infrastructure that supports additional static analyzers and enables analysis of compilable code contexts, with the goal of improving vulnerability detection and memory-safety characterization.

ACKNOWLEDGEMENT

This research was funded in part by U.S. National Science Foundation under Grant Number OIA-2437963 and Louisiana Board of Regents.

References

- [1] Dario Amodei. 2025. *Machines of Loving Grace*. Anthropic Blog. <https://www.anthropic.com/>. Accessed: 2026-06-08.
- [2] Manish Bhatt, Sahana Chennabasappa, Yue Li, Cyrus Nikolaidis, Daniel Song, Shengye Wan, Faizan Ahmad, Cornelius Aschermann, Yaohui Chen, Dhaval Kapil, et al. 2024. Cyberseceval 2: A wide-ranging cybersecurity evaluation suite for large language models. *arXiv preprint arXiv:2404.13161* (2024).
- [3] D. Blain and M. Noiseux. 2026. Broken by Default: A Formal Verification Study of Security Vulnerabilities in AI-Generated Code. *Cobalt AI Technical Report* (2026). [arXiv:2604.05292 \[cs.CR\]](https://arxiv.org/abs/2604.05292)
- [4] Max Brunsfeld et al. 2026. Tree-sitter: An Incremental Parsing Library. <https://tree-sitter.github.io>
- [5] MITRE Corporation. 2026. Common Weakness Enumeration (CWE). <https://cwe.mitre.org>
- [6] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. 2025. Vulnerability Detection with Code Language Models: How Far are We?. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 1729–1741. doi:10.1109/ICSE55347.2025.00038
- [7] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In *2020 IEEE/ACM 17th International Conference on Mining Software Repositories (MSR)*. 508–512. doi:10.1145/3379597.3387501
- [8] GitHub. 2022. GitHub Copilot is generally available. <https://github.blog/2022-06-21-github-copilot-is-generally-available-to-all-developers/>
- [9] GitHub. 2024. GitHub Copilot: 1.8 million paid subscribers. <https://github.blog>
- [10] GitHub. 2025. CodeQL build mode none for C/C++. <https://docs.github.com/en/code-security/how-to-find-and-fix-code-vulnerabilities/manage-your-configuration/codeql-for-compiled-languages>
- [11] Semgrep Inc. 2026. Semgrep: Static Analysis at Ludicrous Speed. <https://semgrep.dev>
- [12] J. R. Landis and G. G. Koch. 1977. The Measurement of Observer Agreement for Categorical Data. *Biometrics* 33, 1 (1977), 159–174.
- [13] Xinghang Li, Jingzhe Ding, Chao Peng, Bing Zhao, Xiang Gao, Hongwan Gao, and Xinchun Gu. 2025. Safegenbench: A benchmark framework for security vulnerability detection in llm-generated code. *arXiv preprint arXiv:2506.05692* (2025).
- [14] Yue Liu, Ratnadira Widyasari, Yanjie Zhao, Ivana Clairine Irsan, Junkai Chen, and David Lo. 2026. Debt behind the ai boom: A large-scale empirical study of ai-generated code in the wild. *arXiv preprint arXiv:2603.28592* (2026).
- [15] Chaomeng Lu, Tianyu Li, Toon Dehaene, and Bert Lagaisse. 2025. ICVul: a well-labeled C/C++ vulnerability dataset with comprehensive metadata and VCCS. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 154–158.
- [16] Jinjun Peng, Leyi Cui, Kele Huang, Junfeng Yang, and Baishakhi Ray. 2025. Cweval: Outcome-driven evaluation on functionality and security of llm code generation. In *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*. IEEE, 33–40.
- [17] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (Copenhagen, Denmark) (CCS '23)*. Association for Computing Machinery, New York, NY, USA, 2785–2799. doi:10.1145/3576915.3623157

- [18] Chihao Shen, Connor Dilgren, Purva Chiniya, Luke Griffith, Yu Ding, and Yizheng Chen. 2026. SecRepoBench: Benchmarking Code Agents for Secure Code Completion in Real-World Repositories. In *Proceedings of the 3rd International Workshop on Large Language Models For Code (LLM4Code '26)*. Association for Computing Machinery, New York, NY, USA, 159–166. doi:10.1145/3786181.3788703
- [19] Mohammed Latif Siddiq and Joanna C. S. Santos. 2022. SecurityEval Dataset: Mining Vulnerability Examples to Evaluate Machine Learning-Based Code Generation Techniques. In *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security (MSR4P&S)*. ACM, 29–33. doi:10.1145/3549035.3561184
- [20] Norbert Tihanyi, Tamas Bisztray, Ridhi Jain, Mohamed Amine Ferrag, Lucas C Cordeiro, and Vasileios Mavroeidis. 2023. The formai dataset: Generative ai in software security through the lens of formal verification. In *Proceedings of the 19th international conference on predictive models and data analytics in software engineering*. 33–43.
- [21] J. Viega, J.T. Bloch, Y. Kohno, and G. McGraw. 2000. ITS4: a static vulnerability scanner for C and C++ code. In *Proceedings 16th Annual Computer Security Applications Conference (ACSAC'00)*. 257–267. doi:10.1109/ACSAC.2000.898880
- [22] Bin Wang, Wenjie Yu, Yilu Zhong, Hao Yu, Keke Lian, Chaohua Lu, Hongfang Zheng, Dong Zhang, and Hui Li. 2025. Ai code in the wild: Measuring security risks and ecosystem shifts of ai-generated code in modern software. *arXiv preprint arXiv:2512.18567* (2025).
- [23] David A. Wheeler. 2026. Flawfinder. <https://dwheeler.com/flipfinder/>
- [24] Tao Xiao, Christoph Treude, Hideaki Hata, and Kenichi Matsumoto. 2024. DevGPT: Studying Developer-ChatGPT Conversations. In *Proceedings of the 21st International Conference on Mining Software Repositories (Lisbon, Portugal) (MSR '24)*. Association for Computing Machinery, New York, NY, USA, 227–230. doi:10.1145/3643991.3648400
- [25] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Curran Associates Inc., Red Hook, NY, USA, Article 915, 11 pages.